

SCENARIO- BASED QUESTION

You are developing a 2D Game System with different characters and actions.

- Create a base class Character:
- Virtual functions attack() and power()

Create derived classes:

- Warrior
- Mage
- (Override both functions)

Create class DamageCalculator:

- Overload calculateDamage():
- 1 parameter → basic attack
- 2 parameters → special attack

Create class Position:

- Data: x, y
- Overload + to combine two positions

In main():

- Use base class pointer to call attack() and power()
- Demonstrate function overloading
- Demonstrate operator overloading

```
#include<iostream>

using namespace std;

class Character{
    public:
        virtual void attack()
        {
            cout<<"\nCharacter attacks!";
        }
        virtual void power()
        {
            cout<<"\nCharacter has a power!";
        }
};

class Warrior: public Character{
    public:
        void attack() override
        {
            cout<<"\nWarrior is also a character so it can attack !";
        }

        void power() override
        {
            cout<<"\nWarrior is also a character so it has a power!";
        }
}
```

```

    };

class Mage: public Character{
    public:
    void attack() override
    {
        cout<<"\nMage is also a character so it can attack !";
    }
    void power() override
    {
        cout<<"\nMage is also a character so it has a power!";
    }
};

class DamageCalculator{
    public:
    void calculateDamage(int x)
    {
        cout<<"\nBasic attack with damage: "<<x;
    }
    void calculateDamage(int x,int y)
    {
        cout<<"\nSpecial attack with damage: "<<x+y;
    }
};

class Position{

```

```

public:
int x,y;
Position operator+(Position const &obj)
{
    Position p1;
    p1.x=x+obj.x;
    p1.y=y+obj.y;
    return p1;
}
};

int main()
{
    Warrior w1;
    Character* characptr=&w1; //run time polymorphism
    characptr->attack();
    characptr->power();

    Mage m1;
    characptr=&m1; //run time polymorphism
    characptr->attack();
    characptr->power();

    DamageCalculator dc;
    dc.calculateDamage(2); //function overloading

```

```
dc.calculateDamage(2,3);
```

```
Position p1;
```

```
p1.x=2;
```

```
p1.y=4;
```

```
Position p2;
```

```
p2.x=1;
```

```
p2.y=7;
```

```
Position p3=p1+p2;
```

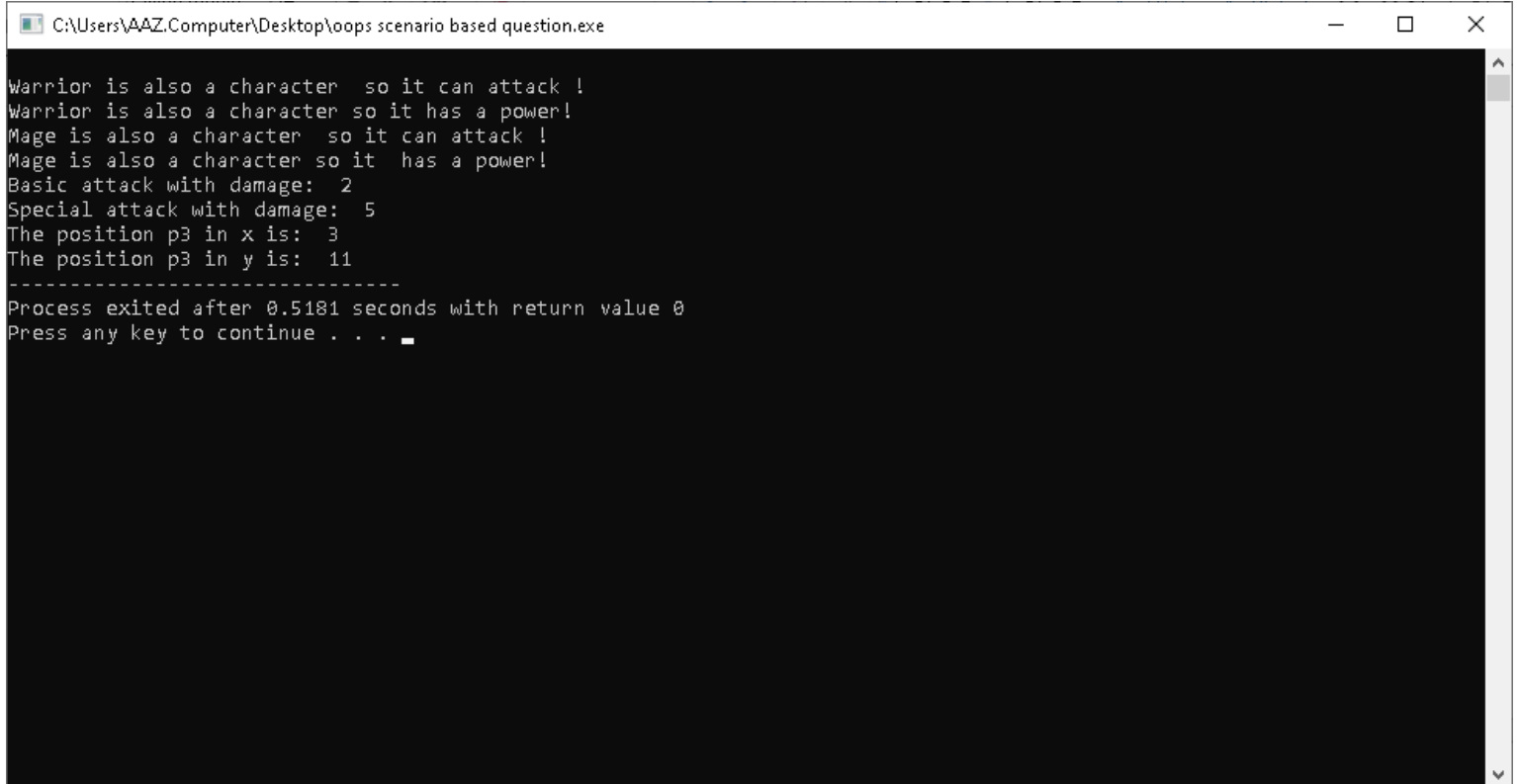
```
cout<<"\nThe position p3 in x is: "<<p3.x;
```

```
cout<<"\nThe position p3 in y is: "<<p3.y; //operator overloading
```

```
return 0;
```

```
}
```

OUTPUT:

A screenshot of a Windows command prompt window. The title bar at the top reads "C:\Users\AAZ.Computer\Desktop\oops scenario based question.exe" and includes standard minimize, maximize, and close buttons. The command prompt area has a black background with white text. The output text is as follows:

```
Warrior is also a character  so it can attack !  
Warrior is also a character so it has a power!  
Mage is also a character  so it can attack !  
Mage is also a character so it  has a power!  
Basic attack with damage: 2  
Special attack with damage: 5  
The position p3 in x is:  3  
The position p3 in y is: 11  
-----  
Process exited after 0.5181 seconds with return value 0  
Press any key to continue . . .
```